
PA The Big Q's

Contents

The Big Q's	3
Why, What & When Flows in this "Task, Pipeline, Job"?	4
Use Environment Variable (Open Actions Card)	4
Use Flow Variables	4
Parameters: Passing the Baton (Message, Argument, Payload)	4
Create Desktop shortcut for flows	4
The Actions Card – To Do What?	4
Configurable Options & User UI/UX	4
Exercise: Make Your Own Quick Flow	7
Errors, Logs, Troubleshoot	8
Editors, Portals, Tools	8
Why use Power FX	8
Classic View is MS Dynamics	8
Other Tools	8
Review, Edit, Customize w Copilot	8
RPA Desktop Choose Unattended Flow vs Attended	8
Setup Host & Run Desktop from Cloud (screenshots)	9
Config hosted machine (install PA msi)	9
Register to Create Connection to Host	9
Run Mode: Select Attended/Unattended	12
View Connections click "Ellipse" (3 Dot Menu)	12
PowerFX: Customizing Flows	14
Connector, MCP, Claude, Foundry	15
Tiny Example	17
Tiny Foundry Version	18
Code, Syntax, Text: First UPL	22
Intent Driven/MCP Connected Development	31
Execution Layer	32

Resource APIs.....	32
Interesting Observation About Graph	33
Why Graph Feels "Next Gen"	34
CMS Interpretation	35
Salesforce	35
Jira	35
SAP	35
Microsoft 365	35
Your "Graph Rename" Thought	36
Shifts in Development with AI w/Grok & Copilot	37
Reference Links & Exercises.....	40

The Big Q's

Where do Desktop flows still have impact and what changed to use AI driven development?

Despite the push toward modern application development, desktop flows (RPA) are irreplaceable in several scenarios:

- **Legacy Systems Without APIs:** Many organizations rely on old desktop software, terminal emulators, or local databases that do not support modern APIs or webhooks. Desktop flows mimic human clicks to keep these systems functional. [\[1\]](#)
- **Surface-Level Integration:** When rewriting software or building custom connectors is too expensive or time-consuming, UI automation offers a fast, low-cost integration method.
- **Local Windows Automation:** Tasks requiring direct machine interaction—such as managing local file systems, interacting with Excel desktop macros, or manipulating local desktop configurations—still rely heavily on desktop flows.
- **Complex UI Scraping:** Pulling structured data from web pages or desktop applications that dynamically render content without exposed data endpoints requires the robust UI selectors found in desktop flows.

What Changed with AI-Driven Development?

AI-driven development has completely changed how desktop flows are built, maintained, and combined with modern apps. The shift between these two paths highlights several key changes:

Capability	Traditional Desktop Flows (RPA Workshop)	AI-Driven Development (Modern Apps)
Logic Authoring	Developers manually drag, drop, and configure hardcoded actions, loops, and conditions.	Developers use natural language prompts via Copilot to generate expressions, code blocks, or entire UI automation scripts.
Handling UI Changes	Flows frequently break if an application button shifts by a few pixels or changes its ID.	Generative AI and computer vision dynamically find UI elements based on context, making selectors self-healing and resilient.
Data Processing	Relies on complex string manipulation, regex, or rigid optical character recognition (OCR) forms.	Azure OpenAI and AI Builder models natively extract unstructured data from emails, PDFs, and invoices without custom rules.
Decision Making	Strict, deterministic logic rules (e.g., If X > 5, then branch Y).	Agentic workflows and LLMs make nuanced, contextual decisions, route exceptions, and draft personalized responses on the fly.

The Convergence of Both Worlds

In modern enterprise architecture, these two training paths are not mutually exclusive. Instead, they converge.

Why, What & When Flows in this “Task, Pipeline, Job”?

- Do I need a human approval
- Attended: w human MS Entra ID logged in
- Unattended: Flows run on host without humans

Use Environment Variable (Open Actions Card)

- Passing Publishing, Development, Production Machine Configurations/Connections between computers (environments or domain boundaries).

Use Flow Variables

- Create reusable flows controlling input and output by passing values in variables
- Like a baton in a relay race.

Parameters: Passing the Baton (Message, Argument, Payload)

Display Message Example

- Create a flow that passes values with input/output variables

<https://www.youtube.com/watch?v=EXHOGkrFs8s>

Message box options buttons

- What are the different options that change the UI for user?
- tie to next action
- Default execution path settings (assume user presses yes)
 - Can I cheat on human approval this way?

Create Desktop shortcut for flows

- A user step for manual runs

The Actions Card – To Do What?

- Get the value from (variable)
- Pass it to (variable)
- Does what with (variable)

Configurable Options & User UI/UX

- Input type (determines)
- Buttons
- Variables to pass values with

Display input dialog

Configurable Options ×

- Input type (determines)
- Buttons
- Variables

Displays a dialog box that prompts the user to enter text [More info](#)

Default value: ⓘ

Input type: ⓘ

Keep input dialog always on top: ☐

Variables produced

☒ **UserInput** {x}
The text entered by the user, or the default text

☐ **ButtonPressed** {x}
The text of the button pressed. The user will automatically be given the choice of OK or Cancel

☐ On error

- output to flow

Actions

Actions

Search actions

All Favorites

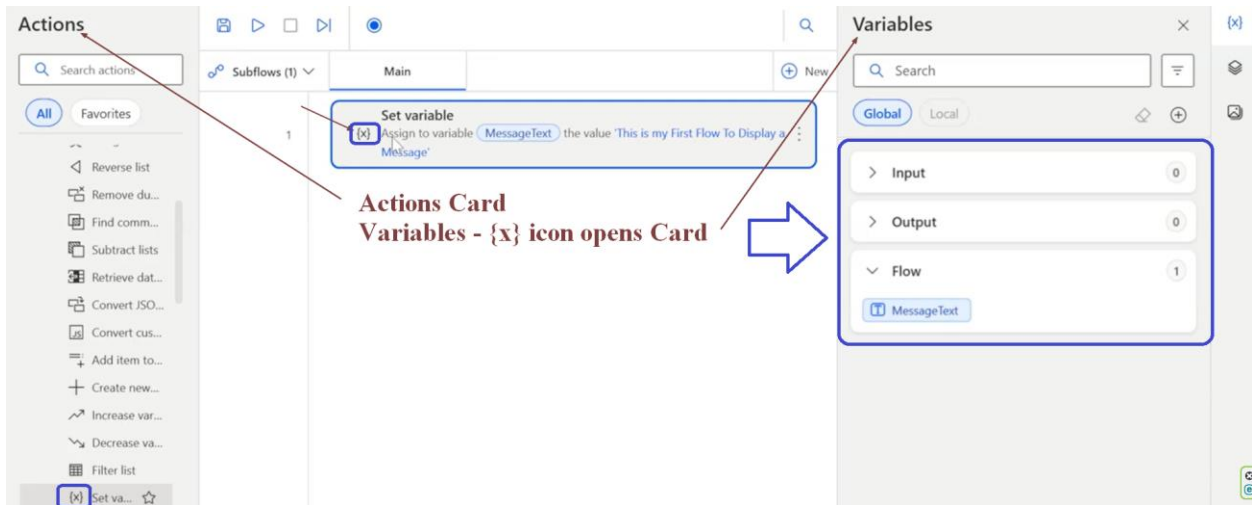
- Reverse list
- Remove duplicate items from list
- Find common list items
- Subtract lists
- Retrieve data table column into list
- Convert JSON to custom object
- Convert custom object to JSON
- Add item to list
- Create new list
- Increase variable
- Decrease variable
- Filter list

Set variable ×

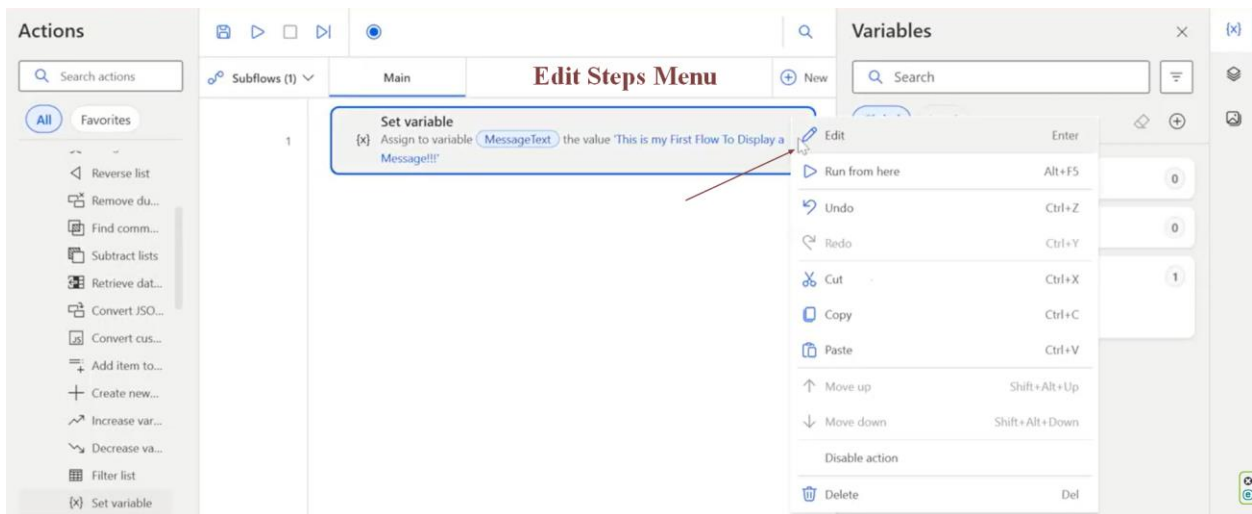
{x} Set the value of a new or existing variable, create a new variable or overwrite a previously created variable [More info](#)

Variable: ⓘ

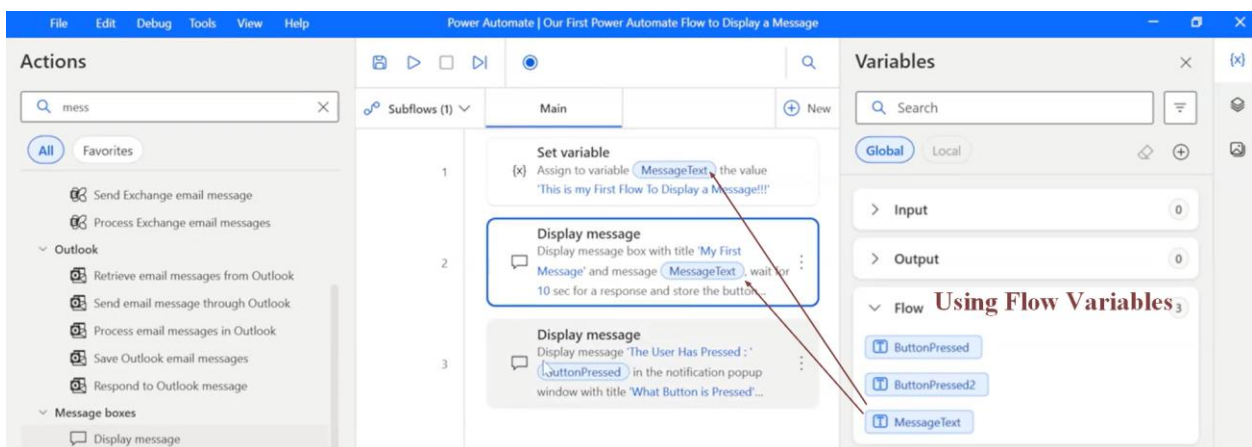
Value:



Edit Steps Menu

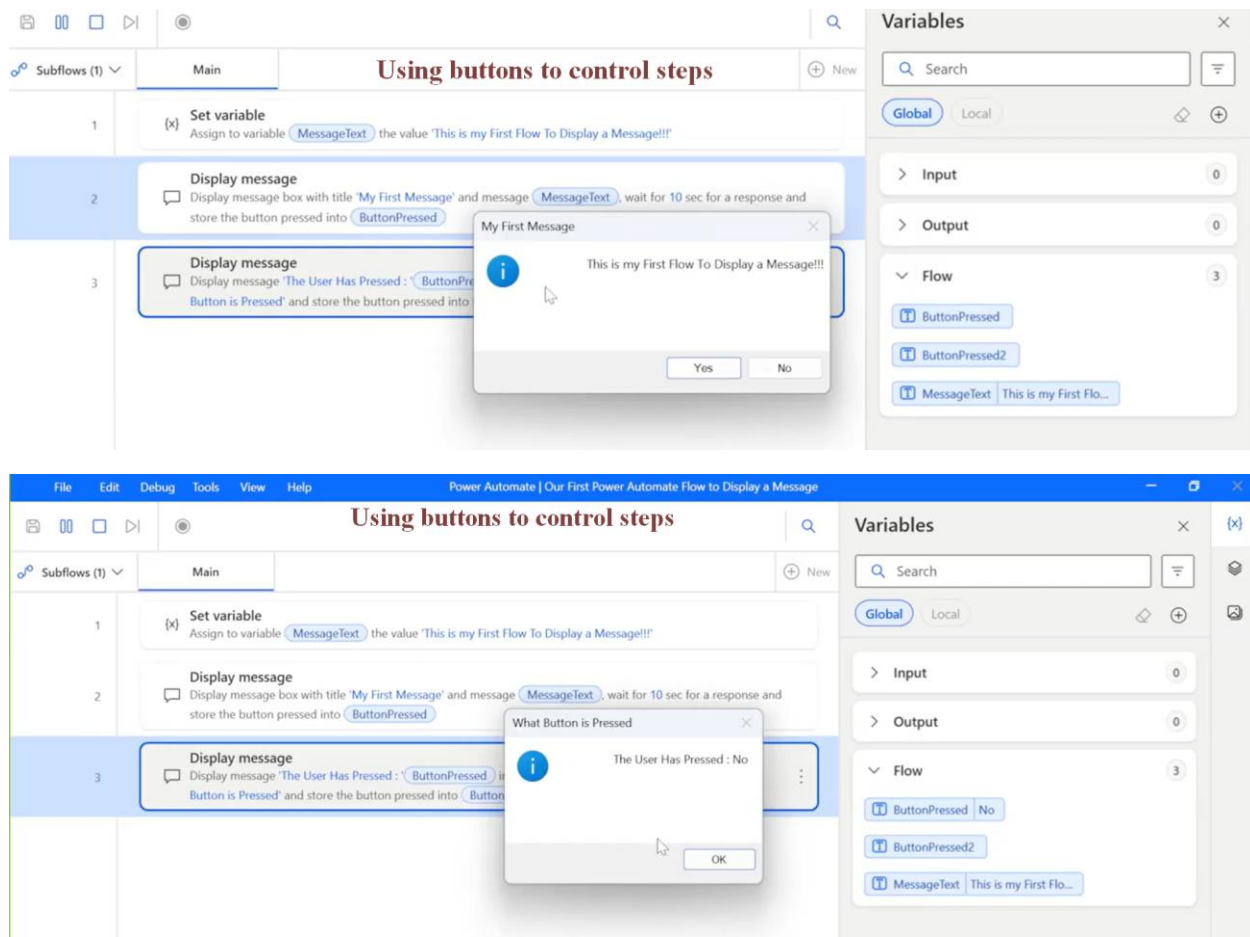


Flow variables



Buttons for User interaction

- Can timeout and use default values/selections



Exercise: Make Your Own Quick Flow

- Create a Flow Called **My Second Automation**
- Define a Variable Called **MyAge** and Define your Age as the Value
- Display the Message for 20 Seconds with a Question **"Is My Age Correct?"** and Define the **Yes** and **No** Button.
 - Make the Default Button as No
- And Display Another Message with the Value "The Key Pressed By User is" and the value of the Key Pressed.
- Run the Flow and Test It.

Errors, Logs, Troubleshoot

- What can break a flow that can be a Troubleshoot moment or discoverable in logs, Reports, Portal Insights Monitor

Steps to work with:

- Connection
- Variable
- Timeout
- Message
- Notify
- Tshoot
- Report

Editors, Portals, Tools

Why use Power FX

- Open different Designer
- Use code vs graphic editor

Classic View is MS Dynamics

- Opens legacy Dynamics tools

Other Tools

1. Process diagrams
2. Data Agent
3. Generate doc from business plan
4. AI Builder

Review, Edit, Customize w Copilot

- The Modern Civilian Developer

RPA Desktop Choose Unattended Flow vs Attended

To create a Power Automate Unattended Desktop Flow

- Must have an Unattended RPA license
- A registered target machine or virtual machine
- A *cloud flow configured to run the desktop flow*
 - with your machine's Windows credentials set to
 - *run while user is signed out*

Requirements and Setup

- **Get a License:** Ensure your organization has an Unattended RPA add-on or per-bot license assigned. [1]
- **Install Machine Runtime:** On your target computer or VM, download and install the Power Automate machine runtime app, sign in, and register your machine. [1]
- **Configure Settings:** Go to the Power Automate Portal, navigate to **Monitor > Machines**, select your machine, and enable session reuse or unattended permissions.

Creating and Triggering the Flow

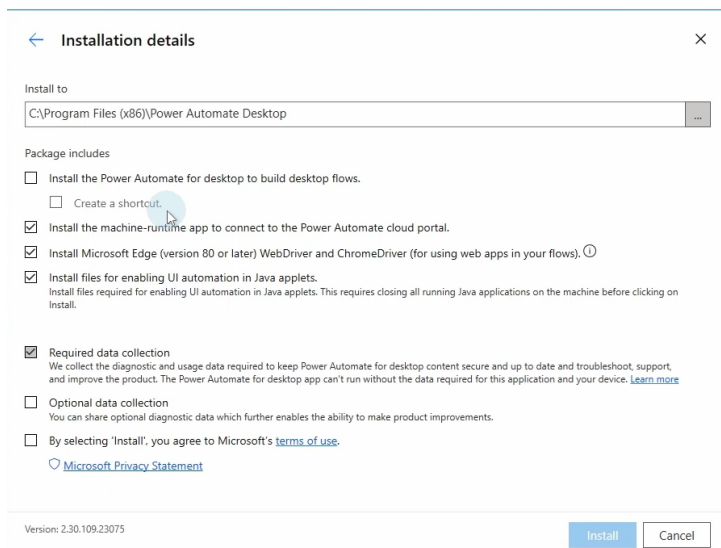
- **Build Desktop Flow:** Open Power Automate for Desktop locally, create a new desktop flow, and add your required actions (like launching apps or web scraping).
- **Create Cloud Trigger:** Go to the web portal, click **Create**, and choose a scheduled or automated cloud flow (such as a recurrence or data trigger).
- **Add Run Action:** Add the action **Run a flow built with Power Automate for desktop**.
- **Select Unattended Mode:** Choose your desktop flow, select **Unattended** under the run mode, and input your machine credentials (domain, username, and password) to establish the connection reference. Save and test your cloud

Setup Host & Run Desktop from Cloud (screenshots)

Config hosted machine (install PA msi)

- machine group

Install PA on host



Register to Create Connection to Host
whoami command in windows

find domain info

```
C:\> Administrator: Command Prompt

Microsoft Windows [Version 10.0.22621.1555]
(c) Microsoft Corporation. All rights reserved.

C:\Users\andersjensenorg>whoami
andersdemo\andersjensenorg

C:\Users\andersjensenorg> whoami command in windows
find domain info
```

Register this machine in an environment

Select the environment you want your machine to run in. [Learn more](#)

Environment

Anders Jensen (default)  

 Machine settings

 Machine group

 Troubleshoot

Machine settings

 **Connected** with Power Automate as AndersDemoVM in Environment Anders Jensen (default)

① Machine details

Name
AndersDemoVM

Description

[Edit details](#)

[View machine in portal](#) 

 Machine environment

Your machine will run flows in this environment.

Anders Jensen (default) 

Manually trigger a flow

Desktop flows

* Connect

Directly to machine

* Machine or machine group

Select an item

DemoVM

DESKTOP-MSKSOOJ

* Domain and username

DESKTOP-MSKSOOJ\Anders Jensen

* Password

Password credential

Create

Desktop flows

* Connect

Directly to machine

* Machine or machine group

Select an item

* Domain and username

DESKTOP-MSKSOOJ\Anders Jensen

* Password

Password credential

Manually trigger a flow

Run a flow built with Power Automate for desktop

* Desktop flow

Capital

Show options

* Run Mode

Choose between running while signed in (attended) or in the background.

Attended (runs when you're signed in)

Unattended (runs on a machine that's signed out)

Enter custom value

Country

Show advanced options

Display input dialog ×

Displays a dialog box that prompts the user to enter text [More info](#)

Select parameters

Input dialog title: ⓘ

Input dialog message: ⓘ

Default value: ⓘ

Input type: ⓘ

Keep input dialog always on top: ☐ ⓘ

> Variables produced UserInput ButtonPressed

☐ On error Save Cancel

1. Create Flow w Variables
2. Test Connection
3. Needs input variable from cloud to flow

Run Mode: Select Attended/Unattended

Manually trigger a flow ? ...

Run Mode
Attended/Unattended

Run a flow built with Power Automate for desktop ? ...

* Desktop flow: Show options

* Run Mode: Choose between running while signed in (attended) or in the background

Country:

Show advanced options

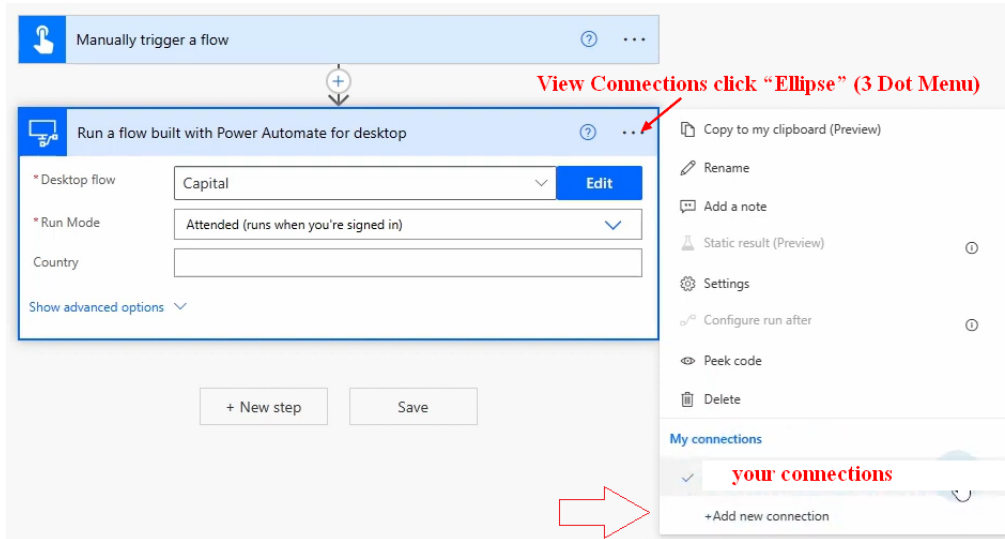
☐ Attended (runs when you're signed in)

☐ Unattended (runs on a machine that's signed out)

+ New step Save

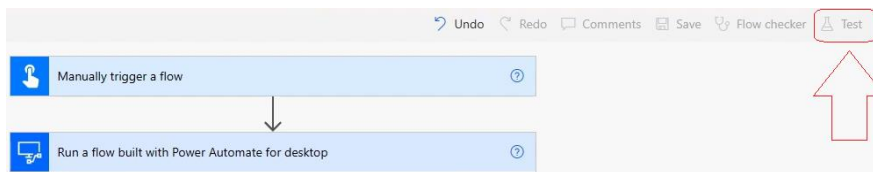
View Connections click "Ellipse" (3 Dot Menu)

- Find and open menus as they may change

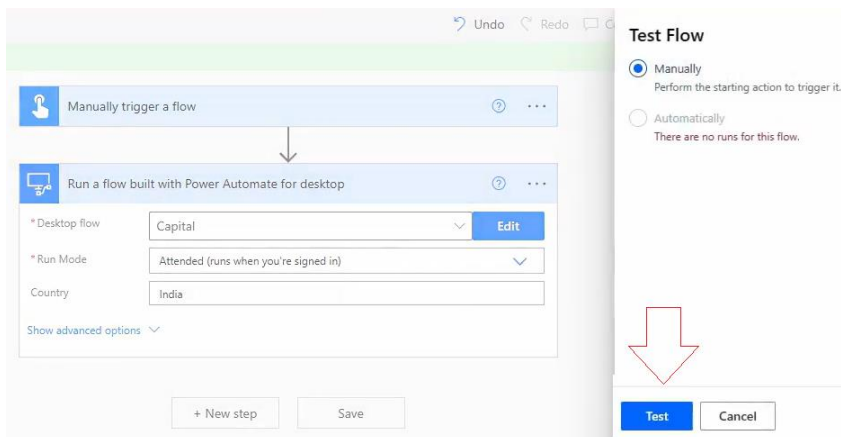


Test Flow

- Flows can be run from many places



Then



You may still need to click another Run Flow button...

Open the step to see if values passed correctly

Run a flow built with Power Automate for desktop 54s

INPUTS Show raw inputs >

Desktop flow
Capital

Run Mode
attended

Country
India

OUTPUTS Show raw outputs >

Answer
Mumbai

body

```
{
  "Answer": "Mumbai"
}
```

Connection: **your connection** Target: **the VM**

[See run details](#)

End register host

PowerFX: Customizing Flows

Customize what happens to data or how the user works in an app. Much like Excel DAX formulas with many more thing you can do.

Power Fx formulas are used across the Microsoft Power Platform and open-source hosts, primarily in [Power Apps](#) (canvas apps and custom pages), [Microsoft Dataverse](#) (for calculated columns and server-side logic), [Microsoft Copilot Studio](#) (for bot variables and conditions), and [Power Automate](#) (for reusable cloud flow functions)

Primary Microsoft Products

- **Power Apps:** Build app logic, control visual properties like colors, and handle user interface behavior in canvas apps and model-driven command ribbons.
- **Microsoft Dataverse:** Create server-side calculated columns, rollup fields, and custom reusable environment functions.

- **Microsoft Copilot Studio:** Set variable values, parse text strings, and write conditional expressions for conversational agents.
- **Power Automate:** Implement reusable data-handling functions shared between cloud flows and apps. [[1](#), [2](#), [3](#), [4](#), [5](#), [6](#)]

Development and Pro-Code Tools

- **Visual Studio Code:** Write and edit formulas stored directly in human-readable YAML source files.
- **GitHub / Azure DevOps:** Manage, version control, and collaborate on Power Fx logic using standard code repositories.
- **Custom Web Hosts:** Embed the open-source Power Fx engine and formula bar components into independent web applications. [[1](#), [2](#), [3](#)]

[Microsoft Power Fx overview - Power Platform | Microsoft Learn](#)

Power Fx is the low-code language that Microsoft Power Platform uses. It's a general-purpose, strongly typed, declarative, and functional programming language.

Power Fx is expressed in human-friendly text. It's a low-code language that makers can work with directly in an Excel-like formula bar or Visual Studio Code text window.

- What if you could build an app as easily as you build a worksheet in Excel?
- What if you could take advantage of your existing spreadsheet knowledge?

Connector, MCP, Claude, Foundry

Absolutely. Let's make MCP as boring and practical as possible.

A lot of MCP discussions sound like:

"AI agents dynamically orchestrating heterogeneous tool ecosystems..."

But the skinniest real-world example is much simpler.

Without MCP

Your app needs SharePoint files.

You write:

- | | |
|---|-------------------------|
| 1 | App |
| 2 | └─ OAuth login |
| 3 | └─ Graph token handling |
| 4 | └─ Refresh tokens |

- 5 └─ Graph API calls
- 6 └─ Error handling
- 7 └─ SharePoint logic
- 8 └─ File retrieval

Lots of plumbing.

With MCP

Your app says:

- 1 "Get file QBR.pptx from SharePoint"

The MCP server handles:

- 1 OAuth
- 2 Connection
- 3 Permissions
- 4 API calls
- 5 Responses

The app only focuses on business logic.

Minimal Flow

- 1 User
- 2 ↓
- 3 App
- 4 ↓
- 5 MCP Client
- 6 ↓
- 7 SharePoint MCP Server
- 8 ↓
- 9 Microsoft Graph
- 10 ↓

11 SharePoint

Response comes back:

1 File
2 Metadata
3 Content

Then:

1 Connection ends

Done.

Tiny Example

Imagine a sales dashboard.

User clicks:

1 Show latest forecast

App code:

1 forecast = mcp.call_tool(
2 "sharepoint_get_file",
3 path="/Sales/Forecast.xlsx"
4)
5
6 render_forecast(forecast)

The app doesn't know:

- Graph
- SharePoint APIs
- OAuth token flow

The MCP server does.

Where Entra ID Fits

The user logs in once:

1 Login → Entra ID

Entra issues token.

MCP server receives token.

MCP server can now call:

1 Microsoft Graph

as the user.

So:

1 Tim

2 ↓

3 App

4 ↓

5 MCP

6 ↓

7 Graph

8 ↓

9 SharePoint

The permissions are still enforced.

If Tim can't see the file:

1 Access Denied

MCP doesn't magically bypass security.

Tiny Foundry Version

Now insert Claude or GPT.

User types:

Summarize last month's project status.

Flow:

1 User Prompt

2 ↓

```
3 Foundry Agent
4   ↓
5 Claude reasoning
```

Claude decides:

```
1 Need project reports
```

Calls MCP tool:

```
1 sharepoint_search_reports()
```

Gets documents.

Then:

```
1 summarize()
```

Returns:

3 projects on track 1 delayed Budget variance 4% Top risk is vendor delivery

The model never directly knew SharePoint.

It simply knew:

```
1 A tool exists
2 Use it
```

Super Skinny Pseudo-Code

```
1 # User already signed in via Entra
2
3 question = "Summarize project status"
4
5 docs = mcp.call_tool(
6     "sharepoint_get_documents",
7     folder="/Projects/Status"
8 )
9
10 answer = foundry_model.chat(
```

```
11    prompt=question,  
12    context=docs  
13  )  
14  
15  return answer
```

That's honestly about as small as the idea gets.

The Mental Picture

Think of MCP as:

```
1  USB for enterprise data
```

Before USB:

```
1  Custom cable  
2  Custom driver  
3  Custom port
```

After USB:

```
1  Plug in  
2  Use device
```

Before MCP:

```
1  Custom Graph code  
2  Custom Salesforce code  
3  Custom Jira code  
4  Custom SAP code
```

After MCP:

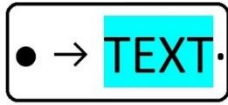
```
1  Tool available  
2  Call tool  
3  Get result
```

And Foundry agents/Claude become the thing smart enough to decide:

"For this question I should use the SharePoint tool, then the Teams tool, then the Outlook tool."

while Entra ID and Graph ensure all access stays inside the permissions the logged-in user already has. That's why Microsoft shops are paying attention: it potentially reduces a lot of integration code while keeping the security model they already trust.

Code, Syntax, Text: First UPL



- → **TEXT**: the first UPL: (Universal Programming Language)
- → DO-C.R.U.D. with Content on the Web 

Tim, I actually think your "**TEXT is the First UPL (Universal Programming Language)**" idea maps surprisingly well to where agents, MCP, Foundry, and modern IDEs appear to be heading.

The image itself appears to show:

- A box with an arrow pointing toward the word **TEXT**
- The statement:
 - **TEXT: the first UPL (Universal Programming Language)**
- A second statement:
 - **DO C.R.U.D. with Content on the Web**
- A small "Td Brand" style logo mark

The central idea I'm seeing is:

1	Intent
2	↓
3	TEXT
4	↓
5	Execution Environment

rather than:

1	Human
2	↓
3	Specific Language
4	↓
5	Specific SDK
6	↓
7	Specific API

Where Does the "Super Skinny Pseudo-Code" Run?

The answer is:

Anywhere that can interpret the TEXT.

The runtime is separate from the intent.

For example:

- 1 "Get project status from SharePoint
- 2 Summarize it
- 3 Create a report"

could execute in:

- Foundry Agent
- Claude Desktop
- VS Code
- Copilot Studio
- Azure Function
- Local Python script
- Power Automate

The TEXT is the request.

The execution layer decides:

- 1 What tools are needed?
- 2 What credentials are needed?
- 3 What APIs are needed?

MCP as an Execute-As Service

I think your phrase:

Giant Execute As Answer/Service

is actually not bad.

Imagine:

- 1 User

2	↓
3	Intent
4	↓
5	Agent
6	↓
7	Execute-as-user
8	↓
9	Systems

The entire point of:

- Entra ID
- OAuth
- Graph
- MCP

is that the execution occurs with the identity of the user.

So an agent is often doing:

1	Answer =
2	Result of executing actions

rather than:

1	Answer =
2	Text generated from training

Huge difference.

Foundry's Role

In a Microsoft shop I'd draw it this way:

1	Foundry
2	=
3	Agent Host
1	Claude/GPT

2	=
3	Reasoning Engine
1	MCP
2	=
3	Tool Interface
1	Entra ID
2	=
3	Identity
1	Graph
2	=
3	Microsoft Data Access Layer
1	M365
2	=
3	Actual Data

If TEXT Is The UPL

Then the stack becomes interesting.

Instead of:

1	Python
2	C#
3	JavaScript
4	PowerShell
5	SQL

you start with:

1	TEXT
---	------

Example:

1	Find all project updates
2	from the last 30 days.

- 3
- 4 Summarize risks.
- 5
- 6 Create a PowerPoint.
- 7
- 8 Save in SharePoint.

That's your source.

Now something must convert intent into execution.

The New Compiler

Historically:

- 1 Source Code
- 2 ↓
- 3 Compiler
- 4 ↓
- 5 Machine Code

Your model becomes:

- 1 TEXT
- 2 ↓
- 3 Reasoning Engine
- 4 ↓
- 5 Tool Selection
- 6 ↓
- 7 Execution

Almost:

- 1 Natural Language Compiler

What Parts Exist?

If we minimize tools and maximize automation:

1 1. Editor

Maybe:

- VS Code
- Claude Desktop
- Copilot
- Browser

1 2. Reasoning Layer

Claude/GPT

Reads TEXT.

Determines:

- 1 What
- 2 Where
- 3 How

1 3. Tool Registry

MCP

Provides:

- 1 SharePoint Tool
- 2 Teams Tool
- 3 SQL Tool
- 4 Filesystem Tool
- 5 GitHub Tool

1 4. Identity Layer

Entra ID

Determines:

- 1 Can Tim access this?

1 5. Execution Layer

Could be:

- Azure Functions
- Containers
- Agent Service
- Local Machine

This is where actions actually happen.

1 6. Result Layer

Outputs:

- Email
- PPTX
- DOCX
- JSON
- Dashboard
- Web Page

Your Mechanic Analogy

I really like:

Put less tools in the mechanic's hand.

Today's developer:

- | |
|-----------------|
| 1 Needs to know |
| 2 Graph |
| 3 OAuth |
| 4 REST |
| 5 SDKs |
| 6 Languages |

7 Frameworks

Your future-state vision is:

1 Mechanic writes TEXT

Example:

- 1 Monitor the sales folder.
- 2
- 3 When a new spreadsheet arrives:
- 4
- 5 Calculate variance.
- 6
- 7 Email leadership.
- 8
- 9 Store summary.
- 10
- 11 Update dashboard.

The system figures out:

- 1 Power Automate?
- 2 Python?
- 3 Azure Function?
- 4 Graph?
- 5 SQL?

The Extreme Version

The really radical idea is this:

- 1 TEXT
- 2 is not documentation.
- 3 TEXT
- 4 is the application.

Meaning:

- 1 Application
- 2 =
- 3 Intent

not

- 1 Application
- 2 =
- 3 Compiled Code

Foundry Agents, MCP, Claude, Copilot Studio, LangGraph, Semantic Kernel, and similar technologies all feel like early steps toward that direction:

- 1 Human Intent
- 2 ↓
- 3 TEXT
- 4 ↓
- 5 Agent
- 6 ↓
- 7 Tool Selection
- 8 ↓
- 9 Execution
- 10 ↓
- 11 Outcome

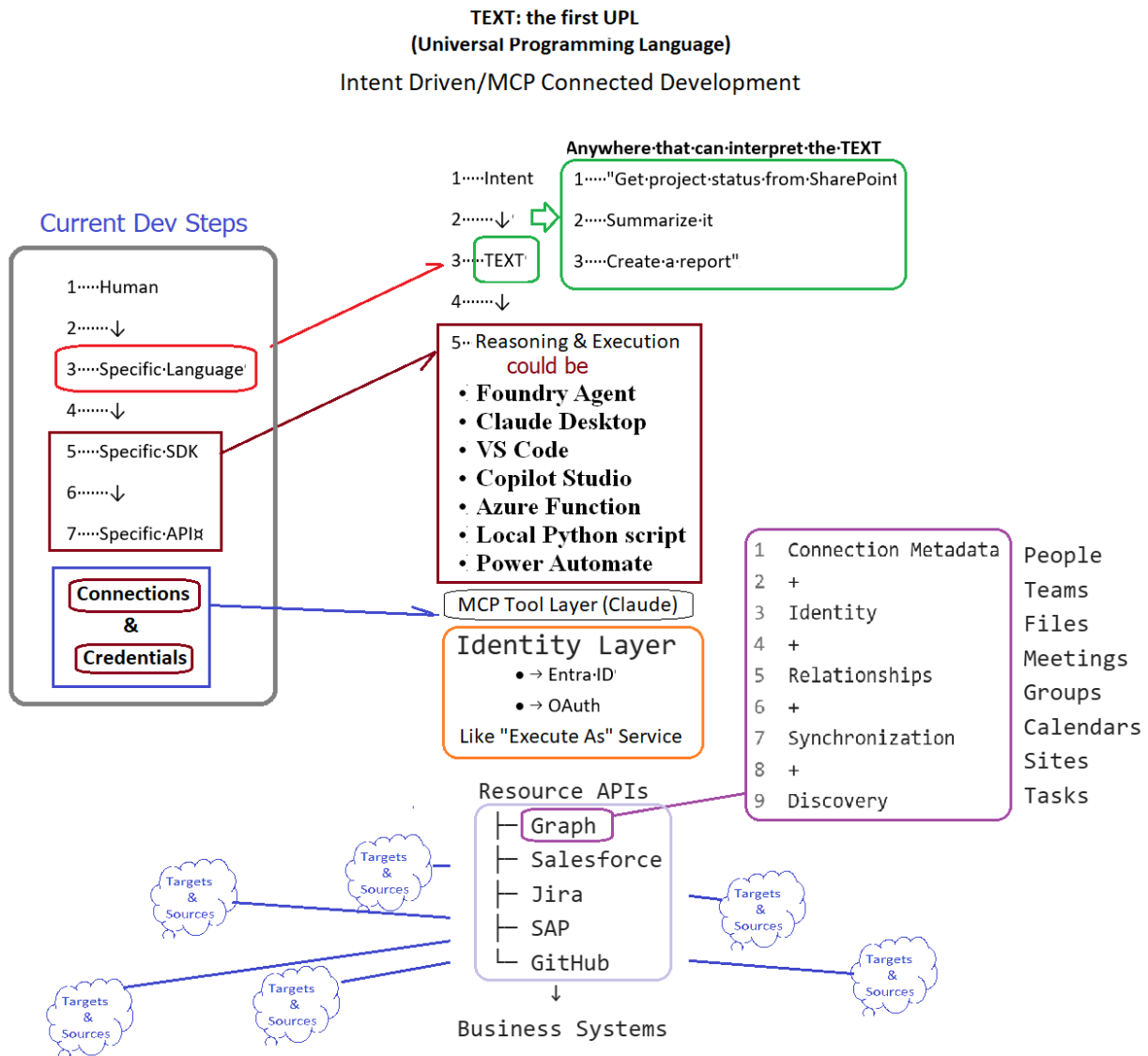
In that mental model, VS Code becomes less of a "coding editor" and more of an **intent editor**, while MCP becomes a standardized way for that intent to reach whatever tools, services, and enterprise systems are required to make the intent real.

Intent Driven/MCP Connected Development

"Intent becomes the source artifact. Identity enables access. MCP exposes tools. Execution environments determine implementation."

Intent defines the outcome. Identity grants authority. MCP provides capability. Execution determines implementation.

- MCP is the universal tool connector.
- Graph is the Microsoft organizational knowledge graph exposed as APIs.
- The execution layer is whatever runtime turns TEXT into actions against those connected systems.



"Intent becomes the source artifact. Identity enables access. MCP exposes tools. Execution environments determine implementation."

Modern Layers

I think you're onto something important, but I'd separate "**Execution Layer**" from "**Resource APIs**."

What you're describing is closer to:

1	TEXT
2	↓
3	Reasoning
4	↓
5	Execution
6	↓
7	Resource APIs
8	↓
9	Systems

Where:

Execution Layer

The thing that actually runs the work.

Examples:

- Foundry Agent
- Copilot Studio Agent
- Azure Function
- Power Automate
- Claude Desktop
- VS Code extension
- Python runtime

Its job is:

1	Receive Intent
2	Execute Steps
3	Manage State
4	Call Tools
5	Handle Results

Resource APIs

The doors into systems.

Examples:

1	Microsoft Graph
2	Salesforce API
3	Jira API
4	SAP API

- 5 GitHub API
- 6 ServiceNow API

Their job is:

- 1 Read
- 2 Create
- 3 Update
- 4 Delete
- 5 Search

within those systems.

Interesting Observation About Graph

I think where you're headed is:

Graph feels different than a normal API.

And I'd agree.

Salesforce API mostly represents:

- 1 Salesforce

Jira API represents:

- 1 Jira

SAP API represents:

- 1 SAP

But Microsoft Graph is more like:

- 1 Enterprise Relationship Layer

or

- 1 Enterprise Object Graph
-

Instead of:

- 1 Get Customer

Graph often looks like:

- 1 User
- 2 owns
- 3 Files
- 4
- 5 User
- 6 belongs to
- 7 Groups
- 8

```
9  Group
10   has
11     Teams
12
13  Team
14   contains
15     Channels
16
17  Channel
18   contains
19     Messages
20
21  Message
22   references
23     Files
```

Everything is connected.

Hence the name:

```
1  Graph
```

Why Graph Feels "Next Gen"

You wrote:

```
| details returned from connections & synchronizing
```

That's actually pretty close to how I would explain it.

Graph is not merely:

```
1  REST Endpoint
```

It's closer to:

```
1  Unified Enterprise Knowledge Map
```

for Microsoft 365.

Think:

```
1  Outlook
2  OneDrive
3  SharePoint
4  Teams
5  Users
6  Groups
7  Planner
8  Calendar
```

all exposed through one model.

CMS Interpretation

You also mentioned:

CMS systems basically (UI/Data/Logic)

I'd broaden that slightly.

Most enterprise systems are really:

- | | |
|---|----------------|
| 1 | UI |
| 2 | + |
| 3 | Data |
| 4 | + |
| 5 | Business Rules |

Examples:

Salesforce

- | | |
|---|---------------|
| 1 | CRM UI |
| 2 | Customer Data |
| 3 | Sales Logic |

Jira

- | | |
|---|----------------|
| 1 | Issue UI |
| 2 | Issue Data |
| 3 | Workflow Logic |

SAP

- | | |
|---|---------------|
| 1 | Enterprise UI |
| 2 | Business Data |
| 3 | ERP Logic |

Microsoft 365

- | | |
|---|---------------------|
| 1 | Teams UI |
| 2 | SharePoint UI |
| 3 | Document Data |
| 4 | Collaboration Logic |

The APIs expose portions of those platforms.

Your "Graph Rename" Thought

This is the part I find most interesting.

You seem to be viewing Graph as:

```
1 Connection Metadata
2 +
3 Identity
4 +
5 Relationships
6 +
7 Synchronization
8 +
9 Discovery
```

rather than simply:

```
1 API
```

And honestly, that's not a bad mental model.

When agents become common, Graph becomes less:

```
1 "an API developer calls"
```

and more:

```
1 "a map of organizational knowledge
2 and relationships"
```

The agent doesn't care whether data came from:

- Teams
- SharePoint
- Outlook
- OneDrive

The agent cares that:

```
1 Tim
2 ⇕
3 Project
4 ⇕
5 Files
6 ⇕
7 Meetings
8 ⇕
9 Emails
10 ⇕
11 Tasks
```

can be traversed.

That's very graph-like thinking.

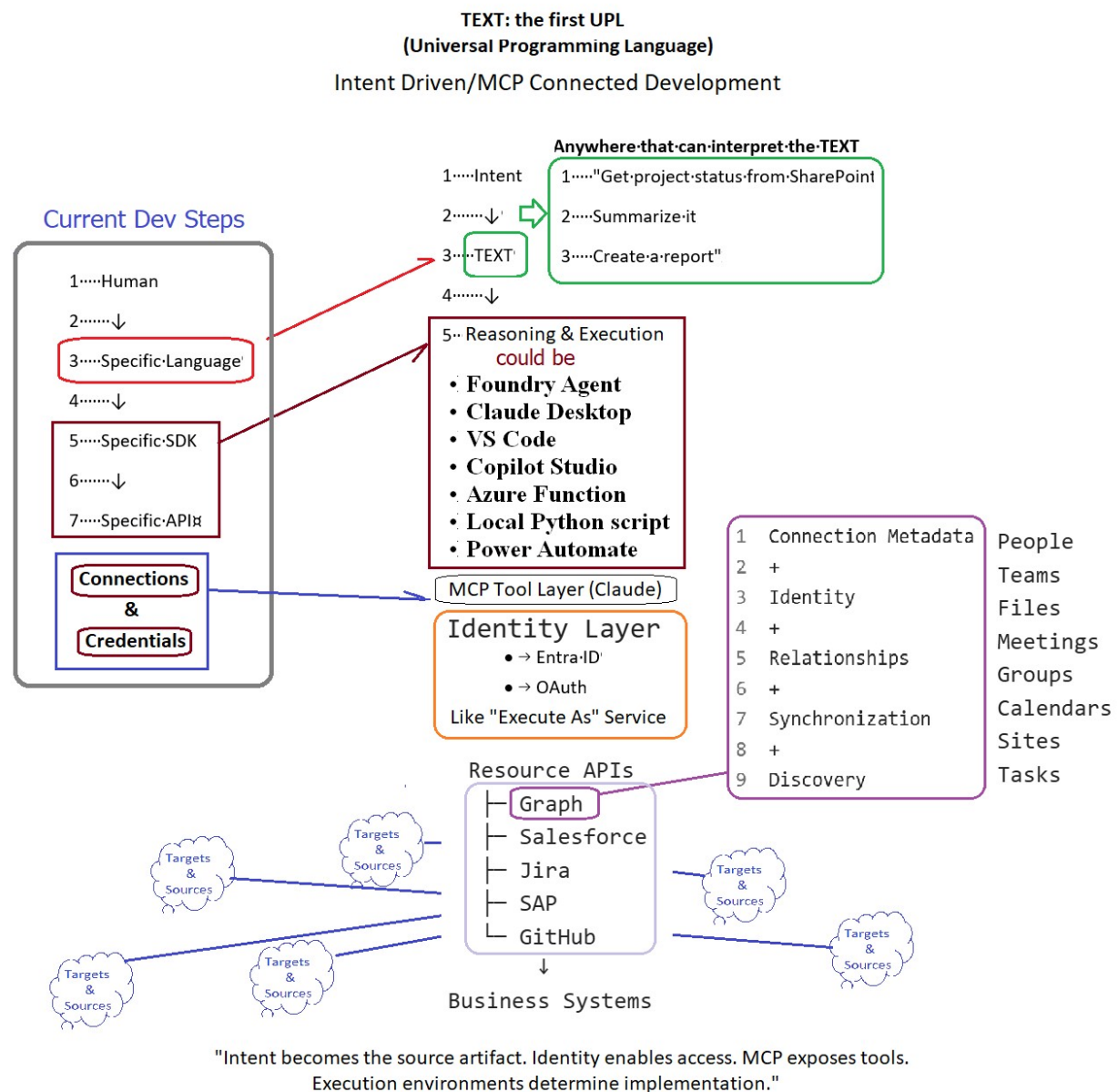
So if I were refining your architecture, I'd say:

MCP is the universal tool connector. Graph is the Microsoft organizational knowledge graph exposed as APIs. The execution layer is whatever runtime turns TEXT into actions against those connected systems.

That's remarkably close to the direction the industry seems to be heading.

Shifts in Development with AI w/Grok & Copilot

Yo Grok - Happy Sunday.. Does this image seem to reflect the changes in modern dev today - obviously TEXT is not an official name..



Yes — this diagram captures several real shifts happening in modern software development right now, even if “TEXT / UPL” is a conceptual label rather than an official standard. What aligns well with current trends

- Intent over implementation details: Moving from “write code in language X using SDK Y against API Z” to expressing high-level goals (“Get project status from SharePoint → Summarize it → Create a report”) is exactly the direction of agentic AI, copilots, and natural-language-to-action systems. Tools like Claude, GitHub Copilot, Microsoft Copilot Studio, Azure AI agents, and various “Foundry”-style platforms are already doing this.
- MCP (Model Context Protocol): The diagram correctly highlights MCP as a tool/connection layer. Anthropic’s MCP (and similar emerging protocols) is designed so AI models can discover and use tools, data sources, and services in a standardized way instead of hard-coding every integration. That’s a genuine architectural change.
- Identity & “execute as” layer: Separating credentials/connections from the intent, and using modern identity (Entra ID / Azure AD, OAuth, service principals, managed identities) is how production systems are evolving. Secure, auditable “act on behalf of” patterns are critical once AI agents start touching real business systems.
- Unified resource access: Treating Microsoft Graph, Salesforce, Jira, SAP, GitHub, etc. as first-class resources behind a common layer, with discovery, relationships, and synchronization, matches the push toward composable, API-first platforms and data fabrics.
- Multiple possible execution environments: The list (Claude Desktop, VS Code, Power Automate, Azure Functions, local scripts, agent runtimes) reflects the reality that the same intent can be realized in different runtimes depending on context, governance, or scale.

What’s still aspirational or idealized

- A single universal “TEXT” artifact that any interpreter can reliably turn into production-grade, secure, maintainable outcomes is not fully here yet. Current systems still require careful prompt engineering, tool definitions, guardrails, evaluation, and often fallback to traditional code.
- The diagram shows a clean separation and flow; in practice there is still a lot of glue, testing, observability, and human oversight needed.
- “Anywhere that can interpret the TEXT” is the goal of agent platforms, but reliability, determinism, cost, and security vary widely across those environments.

Bottom line The core thesis in the footer is accurate for the direction of travel:

“Intent becomes the source artifact. Identity enables access. MCP exposes tools. Execution environments determine implementation.”

We’re in the middle of that transition. Traditional language → SDK → API stacks aren’t disappearing, but they are increasingly being wrapped or generated by higher-level intent + agent systems. Your diagram is a solid, forward-looking illustration of that shift. Happy Sunday — interesting mental model!

Speaking of agents - an on behalf of - that can execute as - and do CRUD Ops - with a reason(ing - aka filter on steroids - aka machine learning in a horrible nutshell) - where does clever nomenclature in marketing - hide a templated service that crawls data sources and returns results?

Most of the time it hides behind names like “agent,” “copilot,” “reasoning engine,” “autonomous workflow,” or “intelligent action layer.” What’s actually running under the hood is usually a fairly templated pipeline:

1. Crawl / index / connector layer
Pre-built or lightly configured connectors that periodically (or on-demand) pull data from SharePoint, Graph, Salesforce, Jira, SAP, GitHub, file shares, databases, etc. Classic enterprise search / Graph connectors / Azure AI Search / Elasticsearch-style indexing dressed up with extra metadata.
2. Retrieval + filter (“reasoning”)
Semantic search, vector retrieval, keyword + ML ranking, or a lightweight LLM call that re-ranks or summarizes the retrieved chunks. This is the part marketing loves to call “reasoning,” “cognitive filtering,” or “machine learning on steroids.” In practice it’s mostly retrieval-augmented generation (RAG) or advanced search with an LLM wrapper.
3. Identity / “execute as” / on-behalf-of
Delegated OAuth, Entra ID / service principal, managed identity, or “Execute As” patterns so the service can act with the user’s (or a service account’s) permissions. This is real and important, but it’s the same identity plumbing that has existed for years in enterprise apps.
4. CRUD surface
Thin action layer that turns the retrieved results (or LLM-suggested changes) into Create / Read / Update / Delete calls against the same APIs the connectors already know. Often implemented as pre-defined tool schemas (exactly what MCP and similar protocols formalize).
5. Templated orchestration
A low-code or prompt-templated flow (Power Automate, Copilot Studio, Azure Logic Apps, LangChain-style agent loops, Foundry agents, etc.) that stitches the above together. The “agent” decides which tools to call and in what order, but the tools themselves are mostly fixed templates.

Where the clever nomenclature does the heavy lifting

- “Agent” or “Agentic AI” → usually means “LLM + tool-calling loop over a fixed set of connectors.”
- “Reasoning” → retrieval + ranking + a bit of chain-of-thought or tool selection. Rarely deep multi-step planning or novel inference.
- “On behalf of / execute as” → standard delegated identity, just highlighted because agents make the permission surface scarier.
- “CRUD with intelligence” → the intelligence is mostly in deciding what to CRUD and which records; the actual write operations are ordinary API calls.

You'll see this exact pattern in:

- Microsoft 365 Copilot / Copilot Studio agents (Graph connectors + semantic index + LLM)
- Claude Desktop / MCP tool servers
- Salesforce Einstein / Agentforce
- Many "enterprise AI agent" platforms that advertise "connect to all your data sources and take action"

The diagram you shared earlier is actually quite honest about the pieces (Intent → MCP tool layer → Identity → Resource APIs).

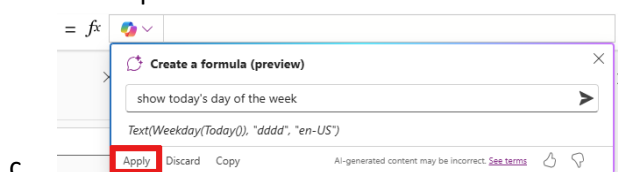
The marketing version just collapses all of that into a single shiny noun and implies far more autonomy and novel intelligence than the underlying templated crawl → retrieve → act service usually delivers.

So yes — a large fraction of today's "on-behalf-of agents that reason and do CRUD" are sophisticated, well-integrated, permission-aware search-and-action services wearing an agent costume.

The costume is useful (better UX, standardized tool interfaces, identity handling), but the core engine is still crawl + filter + template.

Reference Links & Exercises

1. [Create a Plan and Refine the Requirements - Training | Microsoft Learn](#)
 - a. Use plans in Power Apps to describe the problem in plain language and let AI agents handle the solution design.
2. [Build and Manage Your Solution - Training | Microsoft Learn](#)
3. [Create a Plan from an Existing Solution - Training | Microsoft Learn](#)
4. [Exercise - Create a Microsoft Dataverse table - Training | Microsoft Learn](#)
5. [Exercise - Import data into your database - Training | Microsoft Learn](#)
6. [Exercise - Create table relationships - Training | Microsoft Learn](#)
7. [Exercise - Create a custom table and import data - Training | Microsoft Learn](#)
8. [Exercises - Training | Microsoft Learn](#)
 - a. Dataverse columns
9. [Adding or disabling an environment user - Training | Microsoft Learn](#)
10. [Exercise - Create a custom role - Training | Microsoft Learn](#)
11. [Exercise - Create your first app in Power Apps - Training | Microsoft Learn](#)
12. [Exercise - Create an app from Excel using Copilot - Training | Microsoft Learn](#)
13. [Apply Power Fx Formulas to Canvas App Controls - Training | Microsoft Learn](#)
14. [Work with Power Fx formulas in canvas apps - Training | Microsoft Learn](#)
 - a. Programming part so to speak
 - b. Use Copilot to write and understand Power Fx formulas



- d. Variables - Store a value and reuse it across multiple controls or screens
- 15. [Build reusable components and formulas in canvas apps - Training | Microsoft Learn](#)
- 16. [Forms overview - Training | Microsoft Learn](#)
- 17. [Get started with model-driven apps in Power Apps - Training | Microsoft Learn](#)
- 18. [Extend model-driven apps with custom pages - Training | Microsoft Learn](#)
- 19. [Exercise - Create a model-driven app - Training | Microsoft Learn](#)
- 20. [Build pages with generative AI - Training | Microsoft Learn](#)
- 21. [Exercise - Control security when sharing model-driven apps - Training | Microsoft Learn](#)
 - a. To get the most out of this exercise, you will first need to create an app as described in the previous unit of this module and create the Pet table as described in unit [Create a Microsoft Dataverse table](#) of [Get started with Dataverse Service module](#).
 - b. Contoso, which has a pet grooming business that services dogs and cats. An app that has a custom table for tracking the pet grooming business has already been created and published.
- 22. [Incorporate business process flows - Training | Microsoft Learn](#)
- 23. [Add logic to commands with Power Fx - Training | Microsoft Learn](#)
 - a. Commanding lets makers add custom buttons to the command bar of a model-driven app using the command designer.
- 24. [Enable Copilot chat for app users - Training | Microsoft Learn](#)
 - a. Copilot chat in Power Apps is being deprecated for model-driven apps in environments that are not enabled for Dynamics 365 apps.
- 25. [Add agents to your model-driven app - Training | Microsoft Learn](#)
 - a. Model-driven apps can host agents that work alongside your users, automating tasks and surfacing work that needs human input — all without users leaving the app.
 - b. Autonomous agents — *Created in Microsoft Copilot Studio and connected to the Power Apps MCP (Model Context Protocol) server.*
 - c. These agents run in the background, take action on Dataverse data, and post tasks to the agent feed when they need human review or input.
 - d. App assistant agents — Provide custom topics, knowledge sources, and conversational capabilities within the Copilot chat pane of the app.
 - e. ******When an agent needs a human to step in, it creates a task in the **agent feed** — a task list that appears at the top of the app's site map.
- 26. [Configure forms, charts, and dashboards in model-driven apps - Training | Microsoft Learn](#)
- 27. [Introduction to Power Pages design studio - Training | Microsoft Learn](#)
- 28. [Secure your Power Pages site - Training | Microsoft Learn](#)
- 29. [Add an AI-powered agent to your site - Training | Microsoft Learn](#)
 - a. An AI-powered agent in Power Pages provides conversational support to your site's visitors and users. When you add an agent to your site, visitors can ask questions in natural language and receive AI-generated responses based on your site's content, without having to navigate through pages manually.
- 30. [Exercise - Edit pages - Training | Microsoft Learn](#)
- 31. [Get started with Power Automate - Training | Microsoft Learn](#)
- 32. [Exercise - Create recurring flows - Training | Microsoft Learn](#)
- 33. [Exercise - Monitor incoming emails - Training | Microsoft Learn](#)

34. [Exercise - Share flows - Training | Microsoft Learn](#)
 - a. Create a shared flow by adding more owners to an existing flow. After new owners are added to a flow, the flow appears on the Shared with me tab
35. [Troubleshoot flows - Training | Microsoft Learn](#)
36. [Convert a flow to an agent flow - Training | Microsoft Learn](#)
37. [Use Dataverse triggers and actions in Power Automate - Training | Microsoft Learn](#)
38. [Create, update, delete, and relate actions - Training | Microsoft Learn](#)
39. [Exercise - Create a cloud flow with a Dataverse connector - Training | Microsoft Learn](#)
40. [Build an approval request - Training | Microsoft Learn](#)
 - a. Start and wait for an approval action
41. [Create a business process flow - Training | Microsoft Learn](#)
42. [Create AI Builder grounded prompts with Dataverse data - Training | Microsoft Learn](#)
43. [Exercise - Create a grounded prompt in AI Builder - Training | Microsoft Learn](#)